Apprentissage Statistique Automatique II

Practical sessions

Thibault Defourneau

How to use this sheet

Each exercise starts with the sections of the slides and of the lecture notes that it illustrates: the questions turn results of the lecture into experiments.

Exercise	Topic	Status
1	Regularization on synthetic data	Bonus
2	Optimizers, Batch Normalization and residual connections	Core
3	Anatomy of VGG-11	Bonus
4	Equivariance, invariance, and from LeNet-5 to AlexNet	Core
5	Transfer learning and zero-shot classification with CLIP	Bonus
6	Inside a decoder-only language model	Core
7	Semantic search of movies and a mini-RAG	Bonus
8	Grouping movies	Bonus

- **Core and bonus exercises.** Exercises 2, 4 and 6 are the core of the practical sessions. Bonus exercises are optional: they go further on a topic and can be done at home.
- **Environment.** Parts 1 and 2 use TensorFlow with the Keras 3 API, as in the lecture notes. Part 3 uses the library `transformers`, which relies on PyTorch: install `torch` as well. A GPU (for instance on Google Colab) is needed for Exercises 5 and 6, and useful for Exercises 2 and 4.
- **Methodology.** As in the regularization section of the lecture, split the data into training, validation and test sets, select every hyperparameter (architecture, learning rate, regularization strength, number of epochs) on the validation set, and evaluate the final model *once* on the test set.
- **Reproducibility.** Fix the random seeds (`tf.keras.utils.set_random_seed`, `torch.manual_seed`). When two models give close results, rerun the comparison with another seed before drawing a conclusion.
- **Deliverable.** One notebook per exercise, with the requested plots and short written answers (a few lines per question).

1 Training and regularization

Bonus exercise 1 (Regularization on synthetic data)**.**

Course: Regularization: overfitting and methodology, regularization term (ℓ_1, $\ell_2$and AdamW), early stopping, Batch Normalization. TensorFlow: regularizer wrapper.

Generate a binary classification dataset with `sklearn.datasets.make_classification`, using `n_samples=2000`, `n_features=100`, `n_informative=5`, `n_redundant=0`, `flip_y=0.1`, `shuffle=False` and `random_state=42`. Without shuffling, the 5 informative features are the first five columns, and the other 95 features are pure noise.

1. Split the data into 300 training, 700 validation and 1000 test points with `train_test_split`, which shuffles the samples. Standardize the features with statistics computed on the training set only. Why only on the training set?
2. Build an MLP with two hidden dense layers of 256 units with ReLU activations, and an output layer of one unit with a sigmoid activation. Train it with SGD (learning rate 0.01, batch size 32) and the binary cross-entropy for

300 epochs, and plot the training and validation losses. Check that the model overfits; if it does not, reduce the size of the training set.

Show that a softmax output layer with two units, trained with the categorical cross-entropy, defines the same family of classifiers up to a reparametrization.

3. ℓ_2 **penalty.** Retrain the model with `kernel_regularizer=tf.keras.regularizers.L2(lam)` on the hidden layers, for $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$, and select λ on the validation set. For each value, plot the learning curves and report the Euclidean norm of the kernel of the first hidden layer. Which prior on the weights does this penalty correspond to?

4. ℓ_1 **penalty.** Use the class `L1Regularizer` of the lecture notes on the first hidden layer, with the best λ you can find on the validation set. Let $W_1 \in \mathbb{R}^{100\times 256}$ be its kernel. Plot the Euclidean norm of each row of W_1 (one row per input feature) for the ℓ_1 and the ℓ_2 models.

   (a) What do you observe for the five informative features? Relate your observation to the exercise on the geometry of the ℓ_1 and ℓ_2 constraints.

   (b) Are the other weights exactly zero? Why not? (Hint: compare a gradient step on $|w|$ with the soft-thresholding operator of the lecture.)

5. **Early stopping.** Train the unregularized model with the callback `tf.keras.callbacks.EarlyStopping` (`monitor="val_loss"`, `restore_best_weights=True`). Compare its best validation loss with those of Questions 3 and 4. Why is early stopping a form of regularization?

6. **Batch Normalization.** Insert a `BatchNormalization` layer between each hidden dense layer and its activation, without any penalty, and train with batch sizes 8, 32 and 256. How does the gap between the training and validation losses evolve with the batch size? Repeat the experiment without Batch Normalization: which part of the effect is due to Batch Normalization? Relate your answer to the origin of its regularizing effect given in the lecture.

7. Summarize all the models in a table: final training loss, best validation loss, epoch of the best validation loss. Evaluate the model selected on the validation set once on the test set.

Exercise 2 (Optimizers, Batch Normalization and residual connections)**.**

Course: Training neural networks: SGD, momentum, Nesterov, RMSProp, Adam. Regularization: Batch Normalization. CNNs: the degradation problem and ResNet.

We train deep MLPs on CIFAR-10 (60,000 colour images of size 32×32 in 10 classes), loaded with `tf.keras.datasets.cifar10.load_data()`. The network of Question 1 is deliberately too deep for this task: the goal is to observe the difficulties of training deep networks, and the tools of the lecture that address them.

1. **Data and baseline.** Keep the last 5,000 training images as a validation set, and scale the pixels to $[0, 1]$. Build an MLP with 20 hidden dense layers of 100 units with the SiLU activation (also called Swish; `"silu"` in Keras), followed by a softmax output layer. Compute its number of parameters by hand, and check it with `model.summary()`.

2. **Comparison of optimizers.** Train this network with early stopping (`patience=10` on the validation loss, `restore_best_weights=True`) and a batch size of 128, with each of the following optimizers: SGD (learning rate 10^{-2}), SGD with momentum 0.9, SGD with Nesterov momentum, RMSProp (10^{-3}), Adam (10^{-3}) and Nadam (10^{-3}). Plot the validation accuracy against the epochs, and report the best validation accuracy and the number of epochs needed to reach it.

   (a) Nadam is not presented in the lecture: which two methods of the lecture does it combine?

   (b) Which optimizer converges fastest? Which one reaches the best validation accuracy? Discuss your results in the light of the slide "Is Adam always the best choice?".

3. **Momentum.** Compare SGD with learning rate 10^{-2} and no momentum with SGD with learning rate 10^{-3} and momentum $\beta = 0.9$. Relate your observation to the exercise of the lecture showing that $m_\infty = -\frac{\eta}{1-\beta}\, g$ when the gradient is constant. Why are the two learning curves not identical?

4. **Learning rate.** For the best optimizer of Question 2, compare the learning rates 10^{-4}, $3 \cdot 10^{-4}$, 10^{-3}, $3 \cdot 10^{-3}$ and 10^{-2}. What happens when the learning rate is too large? Too small?

5. **Batch Normalization.** Add a `BatchNormalization` layer between each hidden dense layer and its activation, and redo Question 4. The dense layers can then be built with `use_bias=False`: why?

   (a) Does the model converge in fewer epochs? Is each epoch slower? Measure both.

   (b) Is the trained model better on the validation set?

   (c) Can the network now be trained with larger learning rates? Relate your answer to the two explanations of Batch Normalization presented in the lecture.

6. **Degradation and residual connections.** With the functional API of Keras, build a residual MLP without Batch Normalization: a first dense layer of 100 units, followed by 9 residual blocks $y = \mathcal{F}(x) + x$, where $\mathcal{F}$ is made of two dense layers of 100 units and the SiLU activation is applied after the addition (as in the residual block of the lecture), and the softmax output layer.

   (a) Compare the *training* losses of the plain network of Question 1, of the residual network, and of a plain MLP with only 2 hidden layers of 100 units. Does the plain deep network reach a lower training loss than the shallow one? Is this overfitting?

   (b) Explain your observations with the degradation problem presented in the lecture.

7. **Gradients.** On one mini-batch and at initialization, compute with `tf.GradientTape` the norm of the gradient of the loss with respect to the kernel of each hidden layer. Plot it against the depth for the plain network, the network with Batch Normalization and the residual network. Interpret the curves using the Jacobian

$$\frac{\partial y}{\partial x} = I + \frac{\partial \mathcal{F}}{\partial x}$$

   of a residual block.

8. Select your best model on the validation set and evaluate it once on the test set. Summarize in a few lines what makes a deep network trainable.

2 Convolutional Neural Networks

Bonus exercise 3 (Anatomy of VGG-11)**.**

Course: CNNs: convolutional layer and its number of parameters, output size, receptive field, VGGNets, equivariance and invariance (global average pooling), ResNet.

The lecture states that VGG-11 has about 133 million parameters, mostly in its fully connected layers. We check this claim and draw some consequences.

1. Using the table of the VGG-11 architecture in the lecture notes ($224 \times 224 \times 3$ inputs, 1000 classes), compute by hand the number of parameters of each layer, and the total. Build the network with Keras and check your result with `model.summary()`. What proportion of the parameters lies in the fully connected layers?

2. Compute the receptive field of a neuron of the last convolutional block with the recurrence of the lecture, treating each max pooling layer as a layer with kernel size 2 and stride 2. Why is this treatment legitimate? Compare the result with the size of the input.

3. Can VGG-11 be applied without modification to 28×28 images? Justify your answer with the output-size formula.

4. Replace the `Flatten` layer and the two hidden fully connected layers by a `GlobalAveragePooling2D` layer. How many parameters remain? Explain why global average pooling makes the classifier approximately invariant to translations, and why the network can now process images of other sizes.

5. Build the network `ResNet50` of `tf.keras.applications` with `weights=None`, and compare it with VGG-11: number of parameters (method `count_params`) and published top-1 accuracy on ImageNet. What do you conclude?

Exercise 4 (Equivariance, invariance, and from LeNet-5 to AlexNet)**.**

Course: CNNs: convolution and cross-correlation, translation equivariance, effect of the stride, pooling, LeNet-5, AlexNet. Regularization: dropout, data augmentation, Batch Normalization.

Part A: convolution and equivariance.

1. Check your hand computation of the exercise "Example of convolution product" of the slides with `tf.nn.conv2d`, with `padding="VALID"` and with `padding="SAME"`. Which of the two operations of the lecture does this function compute? How can the other one be obtained with the same function?

2. Load Fashion-MNIST with `tf.keras.datasets.fashion_mnist.load_data()`, take one image X and pad it with zeros to the size 48×48 (`tf.pad`). Let Φ be a `Conv2D` layer with 8 filters of size 3×3, stride 1 and `padding="same"`, with random weights. Implement the translations T_τ with `tf.roll`, and compute

$$\max\left|\Phi(T_\tau X) - T_\tau \Phi(X)\right| \qquad \text{for} \qquad \tau \in \{(1,0),\, (2,3),\, (5,-4)\}.$$

   What do you conclude? Why was the image padded with zeros first?

3. Repeat the experiment with a convolutional layer Φ_s of stride 2: compare $\Phi_s(T_\tau X)$ with $T_\delta \Phi_s(X)$ when $\tau = 2\delta$, and check what happens when a component of τ is odd. Then replace the strided convolution by a convolution of stride 1 followed by a 2×2 max pooling layer. Relate your observations to the slide "Effect of the stride".

Part B: from LeNet-5 to a modern CNN. We now classify Fashion-MNIST. Keep the last 10,000 training images as a validation set; the test set is used only once, at the end. Pad the images to the size 32×32, the input size of LeNet-5.

4. **Baseline.** Train the MLP of the dropout practical session of the lecture (`Flatten`, two dense layers of 100 ReLU units, softmax output layer). Report its number of parameters and its validation accuracy.

5. **LeNet-5.** Implement LeNet-5 following the table of the lecture notes, with a softmax output layer instead of the RBF layer, as in the remark of the lecture. Compare its number of parameters and its validation accuracy with those of the MLP. Which properties of convolutional layers explain the difference?

6. **Ablation study.** Starting from LeNet-5, add successively the following innovations of AlexNet presented in the lecture, and report in a table the number of parameters and the validation accuracy of each model:

   (a) ReLU instead of the hyperbolic tangent;

   (b) max pooling instead of average pooling;

   (c) Batch Normalization after each convolution, in place of the Local Response Normalization of AlexNet;

   (d) dropout with rate 0.5 in the fully connected layer;

   (e) data augmentation with random translations of at most 2 pixels (`RandomTranslation`), inserted at the beginning of the model.

   Which modification brings the largest gain? Are all of them useful on this dataset? Like dropout, the augmentation layers are only active during training: why?

7. **Invariance.** For $k \in \{0, 1, 2, 3, 4\}$, translate all the validation images by k pixels to the right, and plot the validation accuracy against k for the MLP, for LeNet-5, and for the best CNN of Question 6 trained with and without data augmentation. Interpret the curves with the notions of equivariance and invariance, and with the interpretation of data augmentation given in the lecture.

8. Evaluate the model selected on the validation set once on the test set.

Bonus exercise 5 (Transfer learning and zero-shot classification with CLIP)**.**

Course: Regularization: data augmentation, Batch Normalization at inference time. CNNs: VGGNets, ResNet. Multimodal LLMs: CLIP and zero-shot classification.

The dataset `tf_flowers` of `tensorflow_datasets` contains 3,670 colour photographs of flowers in 5 classes (daisy, dandelion, roses, sunflowers, tulips). It only provides a `train` split.

1. Load the dataset with `split=["train[:70%]", "train[70%:85%]", "train[85%:]"]` to obtain training, validation and test sets. Check the class balance and display a few images.

2. Build the input pipeline with `tf.data`: resizing to 224×224, preprocessing function of the pretrained model (for instance `tf.keras.applications.resnet50.preprocess_input`), batching and prefetching.

3. **Data augmentation.** Choose a family $\mathcal{G}$ of transformations which do not change the label, and justify each choice: horizontal flips? vertical flips? rotations, and of which amplitude? strong colour shifts? Implement it with

Keras preprocessing layers, and display several augmented versions of the same image. Relate your implementation to the objective

$$\frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_{g\sim\nu}\Big[L\big(f(g\cdot x_i;W),y_i\big)\Big]$$

of the lecture.

4. **Feature extraction.** Load `ResNet50` (or `VGG16`) pretrained on ImageNet without its classification head, freeze it, and add a global average pooling layer and a softmax layer with 5 units. Train this head with and without data augmentation, and compare.

5. **Fine-tuning.** Unfreeze the last block of the pretrained network and continue the training with a learning rate about 10 times smaller. Call the pretrained network with `training=False`, so that its Batch Normalization layers stay in inference mode. Using the slide on Batch Normalization at inference time, explain what could go wrong otherwise.

6. Evaluate the selected model once on the test set and display its confusion matrix.

7. **Zero-shot classification.** Load the CLIP model `openai/clip-vit-base-patch32` with `transformers` and PyTorch. Embed one text per class, for instance "a photo of a daisy, a type of flower", and classify the test images with the rule $\widehat{c} = \operatorname{argmax}_c\, v^\top u_c$ of the lecture, without any training. Compare the accuracy with that of the fine-tuned model. How do the prompts influence the accuracy?